Stage 1 实验报告

于越洋

1 我完成的工作

Step 1

我结合编译器的核心代码，仔细阅读了实验文档中的“通过例子学习”部分，并根据仓库 `README.md` 中的内容进行了测试，以熟悉基本的编译操作。

Step 2

实验框架中已经给出**取负**运算的实现，因此我只需添加**按位取反**和**逻辑非**两种运算。
我在 `utils/riscv.py` 的 `RvUnaryOp` 类中添加了 `NOT` 和 `SNEQ` 两种 RISC-V 指令，在 `utils/tac/tacop.py` 的 `TacUnaryOp` 类中添加了 `BITNOT` 和 `LNOT` 两种操作，分别表示**按位取反**和**逻辑非**。
之后，我在 `frontend/tacgen/tacgen.py` 和 `backend/riscv/riscvasmemitter.py` 的一元操作列表中添加了以上两种运算，以实现从 AST 到 TAC 再到 RISC-V 指令的转换。

Step 3

实验框架中已经给出**加法**运算和**括号**的实现，因此我只需添加**减法**、**乘法**、**除法**、**取模**四种运算。
我在 `utils/riscv.py` 的 `RvBinaryOp` 类中添加了 `SUB` 、 `MUL` 、 `DIV` 、 `REM` 四种 RISC-V 指令，在 `utils/tac/tacop.py` 的 `TacUnaryOp` 类中添加了 `SUB` 、 `MUL` 、 `DIV` 、 `MOD` 四种操作，分别表示**减法**、**乘法**、**除法**、**取模**。
之后，我在 `frontend/tacgen/tacgen.py` 和 `backend/riscv/riscvasmemitter.py` 的二元操作列表中添加了以上两种运算，以实现从 AST 到 TAC 再到 RISC-V 指令的转换。

Step 4

实验框架中已经给出**逻辑或**运算的实现，因此我只需添加**六种比较**运算和**逻辑与**运算。
我在 `utils/riscv.py` 的 `RvBinaryOp` 类中添加了 `SLT` 、 `SGT` 、 `AND` 三种 RISC-V 指令，在 `utils/tac/tacop.py` 的 `TacUnaryOp` 类中添加了 `EQU` 、 `NEQ` 、 `SLT` 、 `LEQ` 、 `SGT` 、 `GEQ` 、 `LAND` 操作，分别表示**等于**、**不等于**、**小于**、**小于等于**、**大于**、**大于等于**、**逻辑与**运算。
之后，我在 `frontend/tacgen/tacgen.py` 和 `backend/riscv/riscvasmemitter.py` 的二元操作列表中添加了以上两种运算，以实现从 AST 到 TAC 再到 RISC-V 指令的转换。
与前两个 Step 不同的是，这里用到的**等于**、**不等于**、**小于等于**、**大于等于**、**逻辑与**运算不能通过一条 RISC-V 指令完成，因此在 `backend/riscv/riscvasmemitter.py` 中，这几种运算我使用多条 RISC-V 指令组合完成。

等于

先用 `sub` 进行相减运算，再用 `seqz` 判断结果是否为 0。

不等于

先用 `sub` 进行相减运算，再用 `snez` 判断结果是否为 0。

小于等于

先用 `sgt` 判断是否大于，再用 `seqz` 对结果取非。

大于等于

先用 `slt` 判断是否小于，再用 `seqz` 对结果取非。

逻辑与

先用 `snez` 判断 `lhs` 是否为 0，将结果写入 `dst`，然后用 `sub` 对 0 和 `dst` 做差，再将结果写入 `dst`，此时若 `lhs` 为 0，则 `dst` 为全 0，否则 `dst` 为全 1（即-1 的补码）。
接下来，使用 `and` 对 `rhs` 和 `dst` 做逻辑与运算，结果写入 `dst`，此时若 `dst` 为 0，说明 `lhs` 和 `rhs` 至少一个为全 0，即数值为 0，否则说明 `lhs` 和 `rhs` 均不为 0。
最后，使用 `snez` 判断 `dst` 是否为 0，已得到 0 或 1 作为最终逻辑与运算的结果。

2 思考题

Stage 0

Step 1

Question 1

> 在我们的框架中，从 AST 向 TAC 的转换经过了 `namer.transform`，`typer.transform` 两个步骤，如果没有这两个步骤，以下代码能正常编译吗，为什么？

```
int main(){
    return 10;
}
```

Answer：能正常编译，因为 `namer.transform` 和 `typer.transform` 分别对应符号表构建和类型检查，该代码不含有变量、函数调用等，不涉及符号表构建，也没有类型错误（实际上，现在的 `typer.transform` 还是一个空壳函数，不包含任何有效操作，即使调用也不会真的执行类型检查），`TACGen` 仅靠 AST 上的 `Return` 语句就能**正确**生成 TAC，完成编译。

Question 2

> 我们的框架现在对于 `return` 语句没有返回值的情况是在哪一步处理的？报的是什么错？

Answer：`return` 语句没有返回值的情况是在 `main.py` 中的 `r: Program = parser.parse(code, lexer=lexer)` 处处理的，`ply_parser.py` 中的 `p_return` 规则只规定了 `statement_matched : Return expression Semi`，若 `Return` 后没有任何 `expression`，则会由 `p_error` 规则处理并添加一个 `DecafSyntaxError` 错误，即显示 `Syntax error` 报错。
使用如下代码

```
int main() {
    return;
}
```

运行 `python3 main.py --input example.c --tac` 后得到的报错信息为

```
Syntax error: line 2, column 12
    return ;
Syntax error: line 3, column 1
}
Syntax error: EOF
```

Question 3

> 为什么框架定义了 `frontend/ast/tree.py:Unary`、`utils/tac/tacop.py:TacUnaryOp`、`utils/riscv.py:RvUnaryOp` 三种不同的一元运算符类型？

Answer：因为 AST 上的一元运算符对应源代码中的一元操作，TAC 上的一元运算符对应三地址码中的一元运算，RISC-V 上的一元运算符对应只有一个操作数的 RISC-V 指令，三者有不同的定义，需要分别定义类型。

Stage 1

Step 2

Question 1

> 我们在语义规范中规定整数运算越界是未定义行为，运算越界可以简单理解成理论上的运算结果没有办法保存在32位整数的空间中，必须截断高于32位的内容。请设计一个 minidecaf 表达式，只使用 `-~!` 这三个单目运算符和从 0 到 2147483647 范围内的非负整数，使得运算过程中发生越界。

Answer：设计的表达式为 `-~2147483647`，因为 `2147483647` 是 32 位补码能表示的最大值，对应 `0x7FFFFFFF`，对其按位取反，得到 `0x80000000`，即 32 位补码能表示的最小值

`-2147483648`，此时对其取相反数会得到 `2147483648`，超出了 32 位补码能表示的范围，从而发生越界。

Step 3

Question 1

> 我们知道“除数为零的除法是未定义行为”，但是即使除法的右操作数不是 0，仍然可能存在未定义行为。请问这时除法的左操作数和右操作数分别是什么？请将这时除法的左操作数和右操作数填入下面的代码中，分别在你的电脑（请标明你的电脑的架构，比如 x86-64 或 ARM）中和 RISCV-32 的 qemu 模拟器中编译运行下面的代码，并给出运行结果。（编译时请不要开启任何编译优化）

Answer：左操作数为 `-2147483648`，右操作数为 `-1`，代码为

```
#include <stdio.h>

int main() {
  int a = -2147483648;
  int b = -1;
  printf("%d\n", a / b);
  return 0;
}
```

在我的电脑（macOS, Apple Silicon, ARM）中运行结果为 `-2147483648`，在 RISCV-32 的 qemu 模拟器中运行结果也为 `-2147483648`。

Step 4

Question 1

> 在 MiniDecaf 中，我们对于短路求值未做要求，但在包括 C 语言的大多数流行的语言中，短路求值都是被支持的。为何这一特性广受欢迎？你认为短路求值这一特性会给程序员带来怎样的好处？

Answer：短路求值主要有两个优势：

1. 提高效率。短路求值可以避免很多不必要的计算，尤其是第二个操作比较复杂的情况下，跳过第二个操作可以节省大量的时间成本。
2. 使代码简洁且安全。比如第二个操作是危险操作（可能出现除以 0、访问空指针等）时，若没有短路求值，需要写成

```
if (a != 0) {
    if (b / a == 1) {
        ...
```

```
    }
}
```

但是有短路求值的情况下就可以简化为

```
if (a != 0 && b / a == 1) ...
```

这样在第一个条件不满足时，确保不会触发第二个条件中的危险操作（除以 0），且代码简洁易懂。